



ОНЛАЙН-ОБРАЗОВАНИЕ

Проверить включена ли запись?



Меня хорошо слышно && видно?



Напишите в чат, если есть проблемы!

Ставьте ☐ + если все хорошо

Ставьте ☐ - если есть проблемы

DAO на Spring JDBC



- Прежние

**Договорились?
Поехали!**

01

Реляционные БД, SQL, H2



Реляционные БД – RDB, RDBMS:

- Хранят нормализованные данные в виде таблиц (отношений)
- Каждая строка имеет первичный ключ
- Одна таблица может ссылаться на первичный ключ другой, с помощью внешнего ключа
- Т.о. таблицы связаны друг с другом

Реляционные БД

ОТЧЕТ

car		
id	number	model_id
1	а001аа 77	2
2	г123ну 99	2
3	р348ву 55	1

Колонка

Таблица



model	
id	name
1	Honda Civic
2	VAZ-21099
3	BMW X7

Таблица

Ячейка

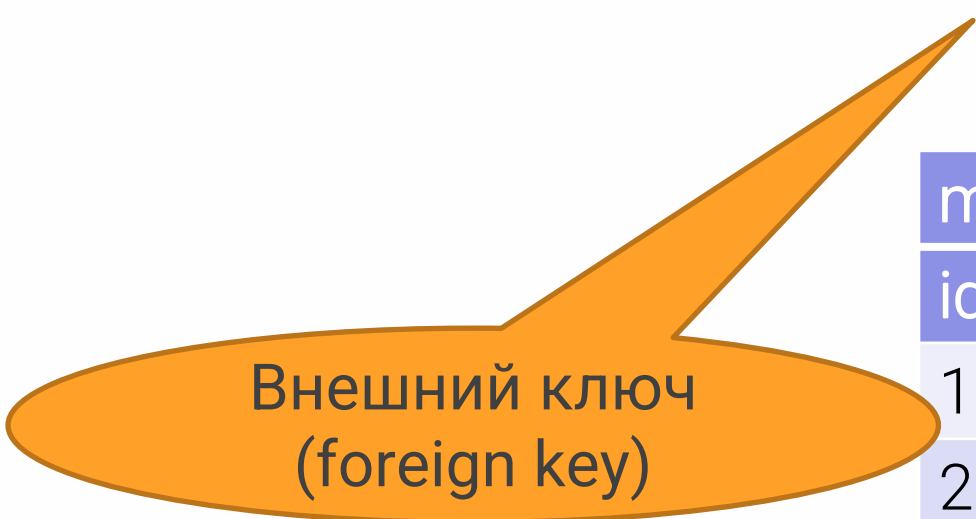
car		
id	number	model_id
1	а001аа 77	2
2	г123ну 99	2
3	р348ву 55	1

Первичный ключ
(primary key)

model	
id	name
1	Honda Civic
2	VAZ-21099
3	BMW X7

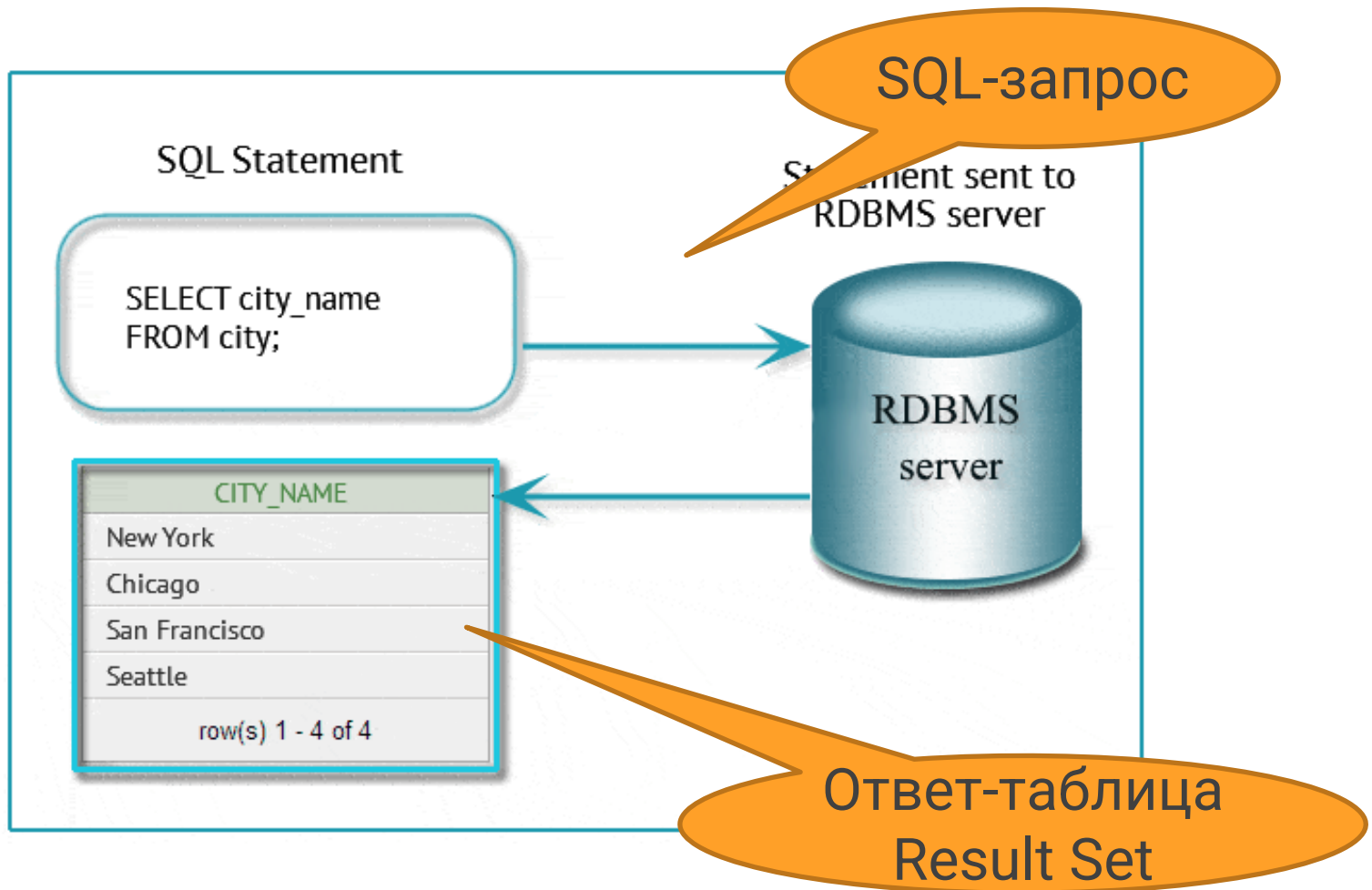
Первичный ключ
(primary key)

car		
id	number	model_id
1	а001аа 77	2
2	г123ну 99	2
3	р348ву 55	1



model	
id	name
1	Honda Civic
2	VAZ-21099
3	BMW X7

- Раньше он быть SEQUEL – Structured English Query Language. Теперь он просто SQL но остался английским
- Изначально предполагалось, что любой человек (менеджер), знающий английский – может написать запрос (нет)
- Для каждой БД – свой диалект SQL – они иногда очень сильно различаются
- Есть общая стандартизированная часть – ANSI SQL, но даже она не во всех БД работает



- SQL Вы можете писать в клиенте БД
- А можете в приложении
- Эти запросы отличаются. Обычно в приложении пишут запросы с параметрами (и их как-то нужно ещё передать БД)
- Ну и в Java-приложении нужна какая-то технология
- Она называется JDBC (про неё чуть попозже)

- БД – очень сложное ПО
- Которое нужно ставить локально
- И в действительности может занимать не один сервер, а несколько – кластер
- А что делать если не хочу, нет ресурсов, не удобно?
- Тут может пригодиться встроенная inmemory БД
- В Java чаще всего используют H2, HSQLDB или Derby, мы будем смотреть на H2

- Откройте консоль H2
- Создайте таблицу TEST
- Добавьте туда запись
- Сделайте select *

```
git clone git@github.com:OtusTeam/Spring.git  
или  
git clone https://github.com/OtusTeam/Spring.git
```

```
cd ./<имя группы>/spring-09/jdbc-class-work
```

Не забудьте указать jdbc:h2:mem:testdb

Подсказка: есть ссылка внизу страницы консоли

Вопросы?

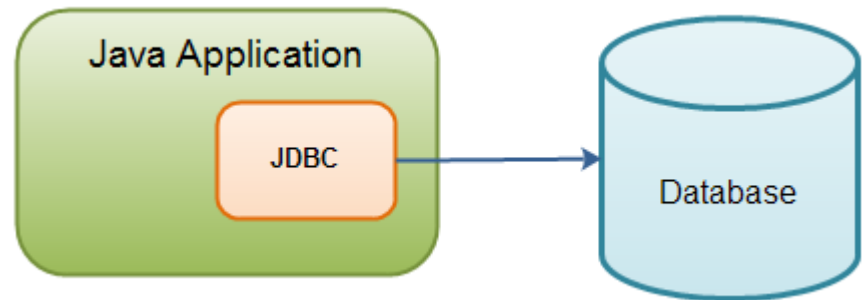
(ставим “+” если вопросы есть и “-”
если вопросов нет)

02

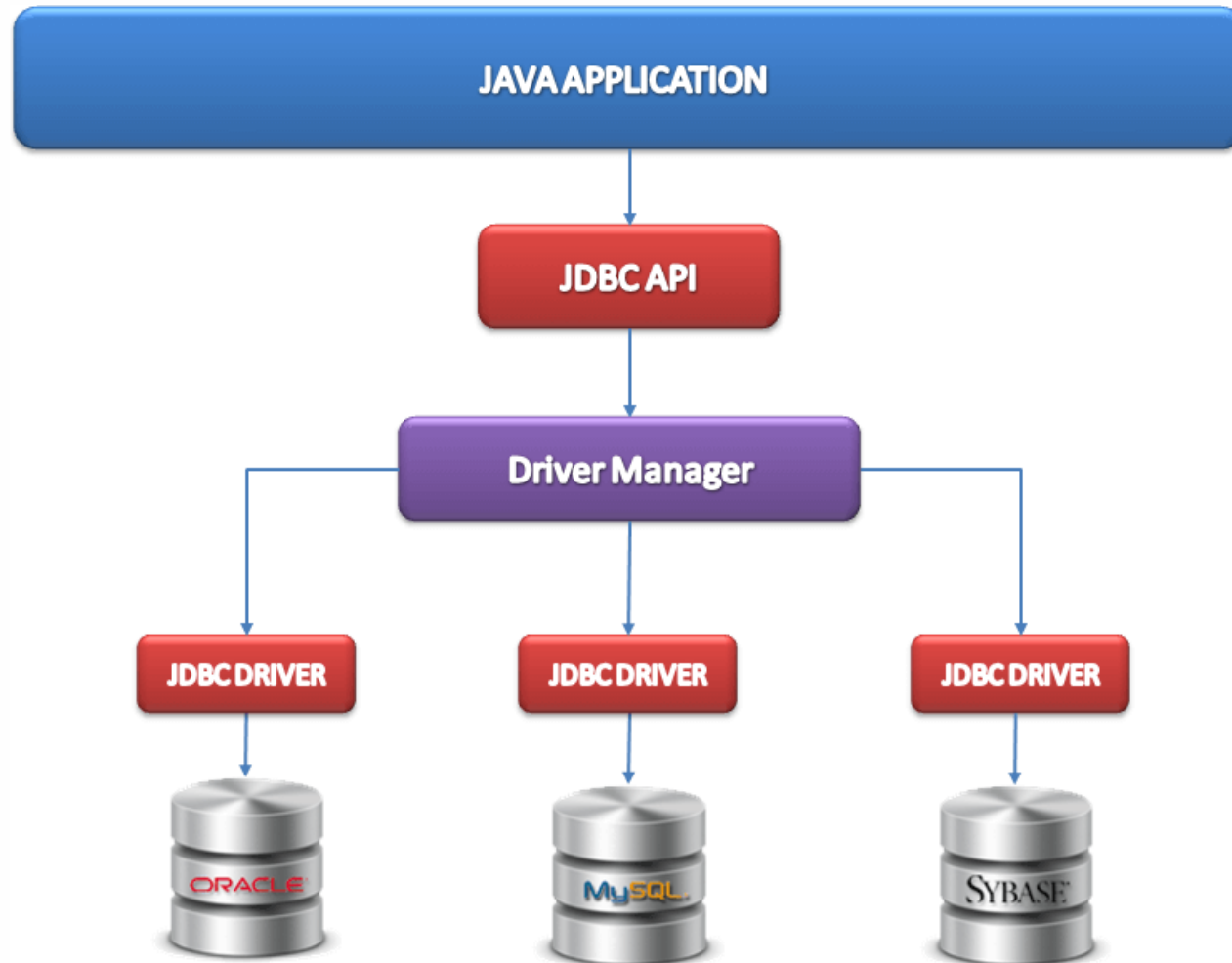
JDBC, Паттерн DAO



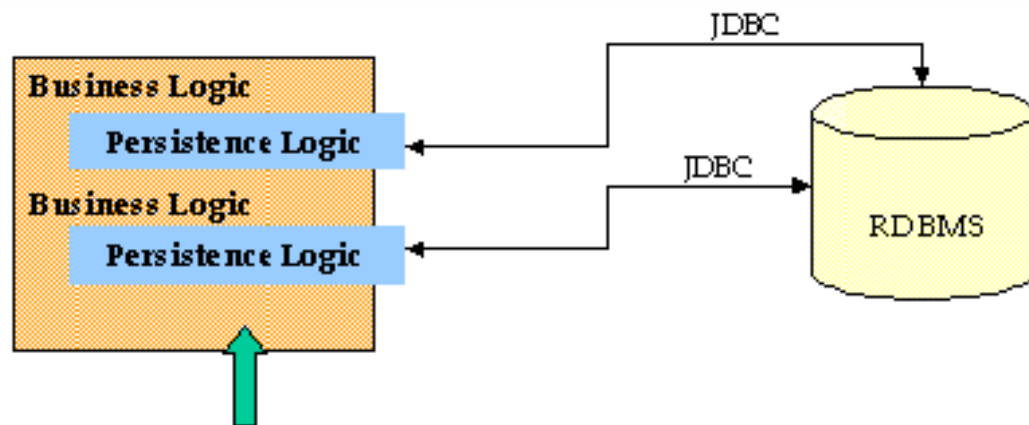
- JDBC хитрая штука
- Которая позволяет абстрагироваться от БД



- Для этого используется такое понятие как драйвер, драйвер выпускают разработчики БД
- Но SQL для каждой БД – свой
- P.S. Код JDBC мы писать не будем

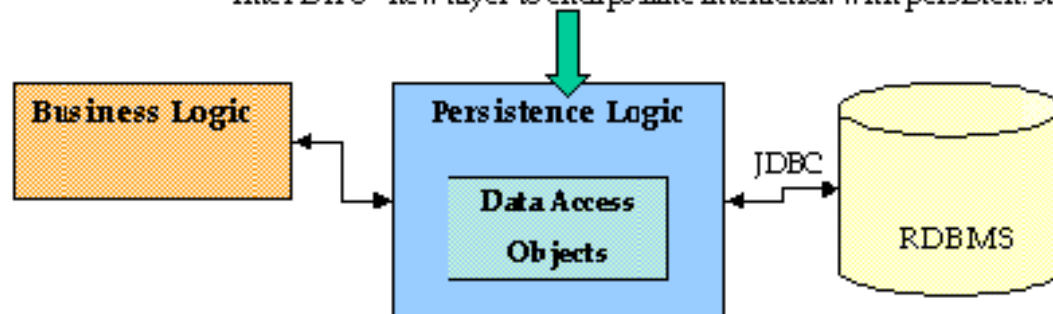


- DAO – Data access object
- Прослойка между бизнесом и БД
- Обычно пишется для каждой БД отдельно
- Ну и соответствует бизнес-сущностям
- В терминах ORM её называют репозиторием



Before DAO—persistence code scattered within business logic.

After DAO—new layer to encapsulate interaction with persistent store



```
public interface PersonDao {  
    int count ();  
    void insert (Person person);  
    Person getByld (long id);  
    List<Person> getAll ();  
}
```

- Иметь доступ к БД (url, login/pass)
- JDBC и так есть в Java
- А вот драйвер добавить надо в зависимости
- Пишем DAO, с SQL запросами в строчку
- И написать код JDBC, подключающийся к БД и отправляющий данный запрос

Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)

03

Spring JDBC



- Чистый JDBC – достаточно старая технология
- И неудобная
- Пользоваться ей напрямую – целый ад

- SQL Exception – checked и одно на все ошибки
- Нужно открывать/закрывать
- Дурацкий PreparedStatement
- Нужно обходить результирующий набор данных “руками”

```
// Plain JDBC
final Connection connection = ds.getConnection();
try {
    final Statement statement = connection.createStatement();
    try {
        final ResultSet resultSet = statement.executeQuery("select count(*) from orders");
        try {
            resultSet.next();
            final int c = resultSet.getInt(1);
        } finally {
            resultSet.close();
        }
    } finally {
        statement.close();
    }
} finally {
    connection.close();
}

//Spring
int c = jdbcTemplate.queryForInt("select count(*) from orders");
```

- **JdbcTemplate (интерфейс – JdbcOperations)** – центральный класс для работы с БД. Выполняет SQL-запросы, итерирует по результатам, ловит JDBC исключения. Threadsafe. Для работы ему необходимы:
 - DataSource
 - RowMapper
 - Собственно, SQL-запрос
- **javax.sql.DataSource** – управляет подключениями, является частью спецификации JDBC, позволяет абстрагироваться от connection-ов и пулов
- **RowMapper** – маппит строчку ResultSet на объект
- При использовании Spring Boot, DataSource и JdbcTemplate будут созданы автоматически, если подключен соответствующий стартер

@Repository

```
public class PersonDaoJdbc implements PersonDao {
```

```
    private final JdbcTemplate jdbc;
```

```
    public PersonDaoJdbc (JdbcTemplate jdbc) {
```

```
        this.jdbc = jdbc;
```

```
    }
```

@Override

```
    public int count () {
```

```
        return jdbc.queryForObject("select count(*) from persons", Integer.class);
```

```
    }
```

```
}
```

```
@Override
```

```
public void insert (Person person) {  
    jdbc.update("insert into persons (id, name) values (?, ?)",  
        person.getId(), person.getName());  
}
```

```
@Override
```

```
public void update (Person person) {  
    jdbc.update("update persons set name = ? where id = ?",  
        person.getName(), person.getId());  
}
```

```
@Override
```

```
public void deleteById (long id) {  
    jdbc.update("delete from persons where id = ?", id);  
}
```



```
@Override  
public void createPersonsTable () {  
    jdbc.execute("create table persons (id bigint primary key, " +  
        "name varchar(255))");  
}
```

- Интерфейс из `org.springframework.jdbc.core`
- Он описывает маппинг строчек `ResultSet` в конкретные объекты
- Используется в методе `query()` `JdbcTemplate`-а или в результате вызова хранимой процедуры
- Обычно сохраняется в поле `DAO`, если он `stateless`



```
@Override
```

```
public Person getByld (long id) {  
    return jdbc.queryForObject("select id, name from persons where id = ?",  
        new Object[] {id}, new PersonMapper()  
    );  
}
```

```
@Override
```

```
public List<Person> getAll () {  
    return jdbc.query("select id, name from persons", new PersonMapper());  
}
```

```
private static class PersonMapper implements RowMapper<Person>{  
    @Override  
    public Person mapRow (ResultSet resultSet, int i) throws SQLException {  
        long id = resultSet.getLong("id");  
        String name = resultSet.getString("name");  
        return new Person(id, name);  
    }  
}
```

```
@Override
```

```
public Person getByld (long id) {  
    return jdbc.queryForObject("select id, name from persons where id = ?",  
        new Object[] {id}, new PersonMapper()  
    );  
}
```

```
@Override
```

```
public List<Person> getAll () {  
    return jdbc.query("select id, name from persons", new PersonMapper());  
}
```

Реализовать метод count в PersonDaoJdbc

```
git clone git@github.com:OtusTeam/Spring.git  
или  
git clone https://github.com/OtusTeam/Spring.git
```

```
cd ./<имя группы>/spring-09/jdbc-class-work
```

Или сразу смотрим решение в проекте jdbc-demo-solution-3



Реализовать метод insert в PersonDaoJdbc

```
git clone git@github.com:OtusTeam/Spring.git  
или  
git clone https://github.com/OtusTeam/Spring.git
```

```
cd ./<имя группы>/spring-09/jdbc-class-work
```

Или сразу смотрим решение в проекте jdbc-demo-solution-4



Написать RowMapper

Реализовать методы getByld и getAll в PersonDAOJdbc

```
git clone git@github.com:OtusTeam/Spring.git  
или  
git clone https://github.com/OtusTeam/Spring.git
```

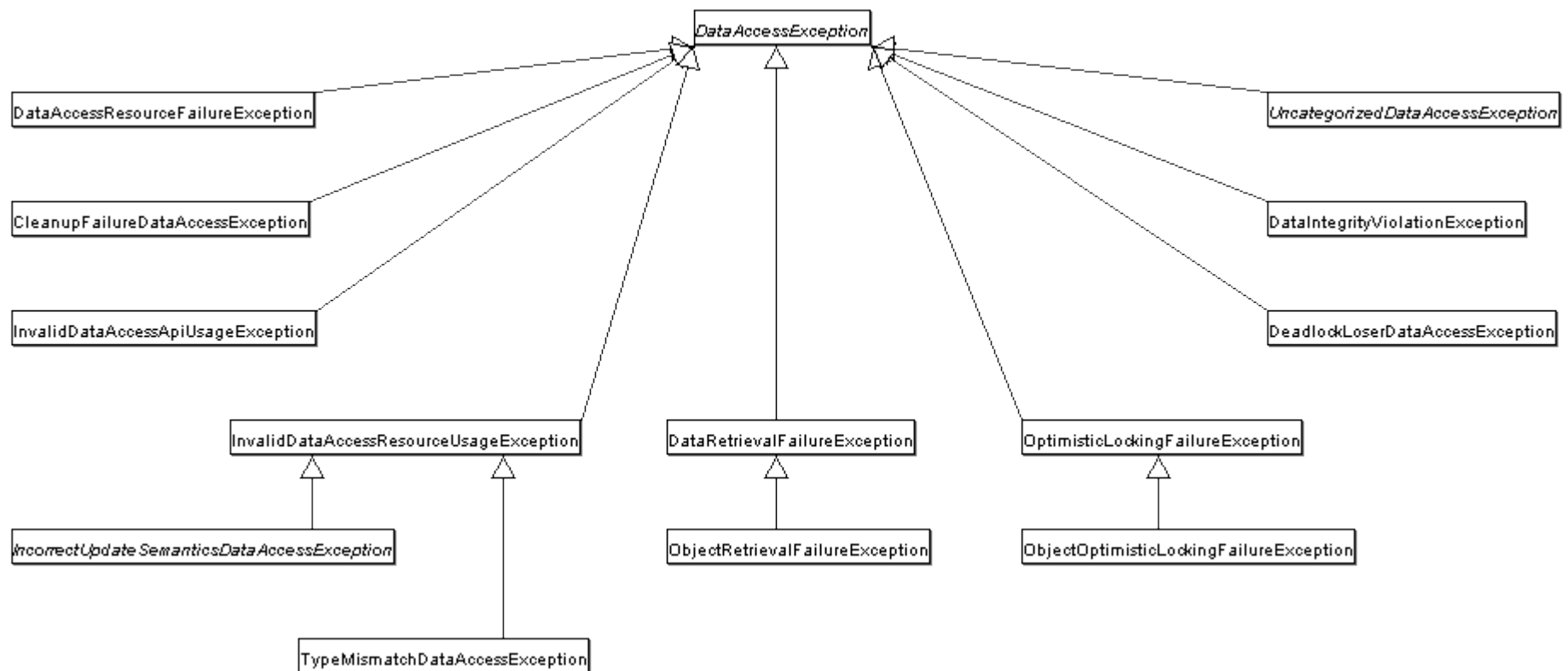
```
cd ./<имя группы>/spring-09/jdbc-class-work
```

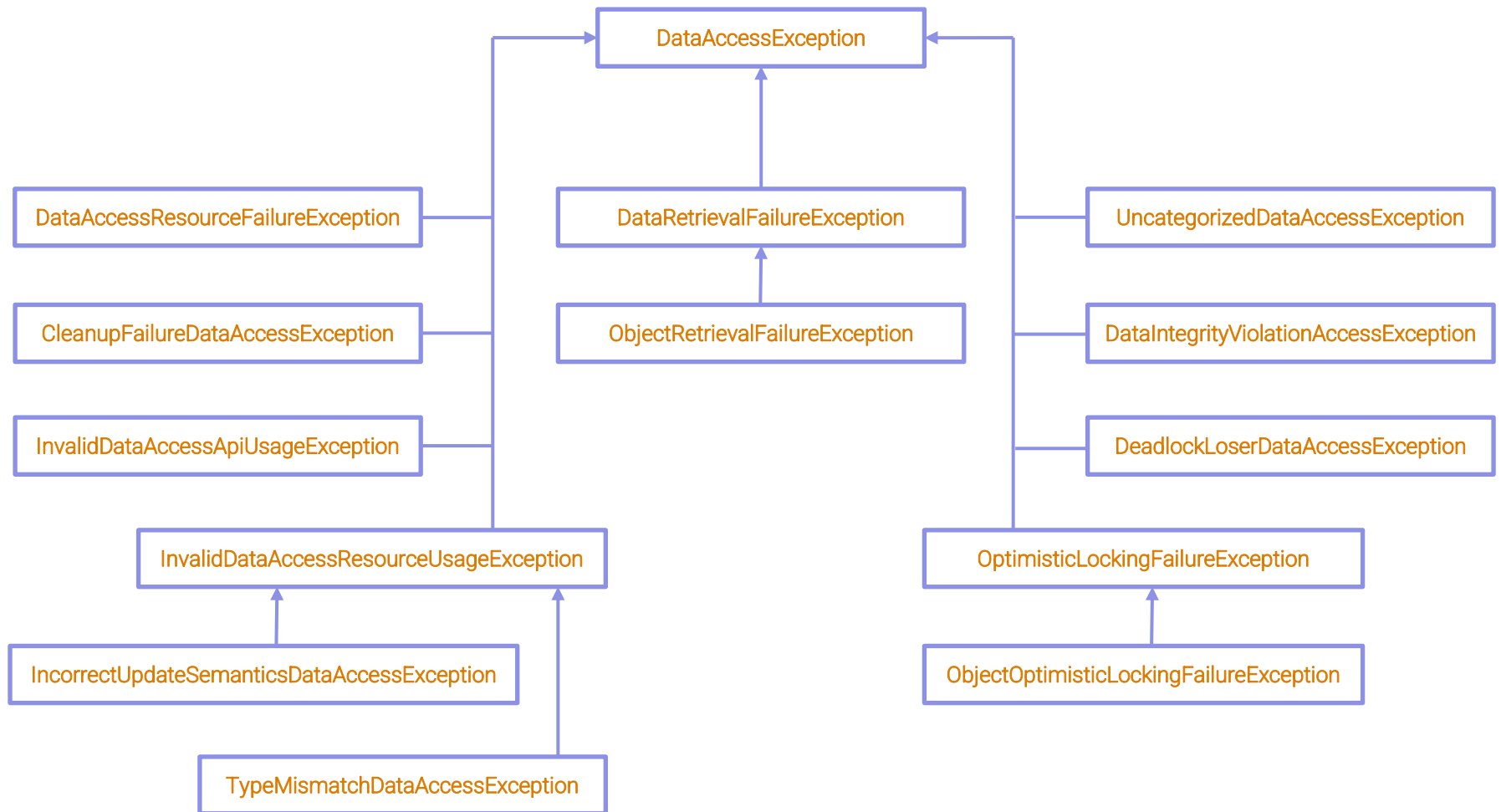
Или сразу смотрим решение в проекте jdbc-demo-solution-5



- Spring преобразует исключения, зависящие от платформы, такие как `SQLException` в иерархию собственных исключений, `DataAccessException` – корень этой иерархии;
- Это runtime-исключения (!)







- Дает возможность задавать параметры не по индексам, а по именам
- Дает доступ к обычному JdbcTemplate через метод
- NamedParameterJdbcOperations – интерфейс – используйте его (можно мокать)
- Использовать только так



@Repository

```
public class PersonDaoJdbc implements PersonDao {  
    private final NamedParameterJdbcOperations jdbc;  
  
    public PersonDaoJdbc (NamedParameterJdbcOperations jdbc) {  
        this.jdbc = jdbc;  
    }  
  
    @Override  
    public Person getByld (long id) {  
        final Map<String, Object> params = new HashMap<>(initialCapacity: 1);  
        params.put("id", id);  
        return jdbc.queryForObject("select id, name from persons where id = :id",  
            params, new PersonMapper()  
        );  
    }  
}
```

Переделать метод getByld и реализовать deleteByld на NamedParameterJdbcOperations

```
git clone git@github.com:OtusTeam/Spring.git
или
git clone https://github.com/OtusTeam/Spring.git

cd ./<имя группы>/spring-09/jdbc-class-work

// можно сделать так
jdbc = namedParameterJdbcOperations.getJdbcOperations();

Или сразу смотрим решение в проекте
jdbc-demo-solution-final
```



Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)

- Чаще всего мы все же не вставляем id из приложения
- Обычно значение поля-идентификатора генерирует БД. Например с помощью автоинкрементного поля (типа BIGSERIAL)
- А знать id только, что вставленной записи иногда очень нужно
- В Spring-JDBC за получение сгенерированного базой id отвечает KeyHolder
- А точнее его реализация: GeneratedKeyHolder




```
@Override
```

```
public long insert (Person person) {
```

```
    MapSqlParameterSource params = new MapSqlParameterSource();  
    params.addValue("name", person.getName());
```

```
    KeyHolder kh = new GeneratedKeyHolder();
```

```
    jdbc.update("insert into persons (name) values (:name)", params, kh,  
        new String[]{"id"} );
```

```
    return kh.getKey().longValue();
```

```
}
```

04

Spring Boot Starter JDBC



```
<dependencies>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-jdbc</artifactId>
    </dependency>

    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <version>${версия.h2.например.1.4.200}</version>
    </dependency>

</dependencies>
```

- Поднимает DataSource
- Создает JdbcOperations
- И NamedParameterJdbcOperations
- Подключает транзакционность
- Автоматически выполняет файлы `schema.sql`, `data.sql`

- Настраивается в `application.yml`
- Если используется H2, HSQLDB или Derby, то ничего писать совсем не надо (нет)

```
spring:
  datasource:
    url: jdbc:mysql://localhost/test
    username: dbuser
    password: dbpassword
    driver-class-name: com.mysql.jdbc.Driver
    #initialization-mode: always
    #schema: schema.sql
    #data: data.sql
  sql:
    init:
      mode: always
      data-locations: data.sql
      schema-locations: schema.sql
```

Смотрим в ресурсы (schema.sql, data.sql)

```
git clone git@github.com:OtusTeam/Spring.git  
или  
git clone https://github.com/OtusTeam/Spring.git  
  
cd ./<имя группы>/spring-09/jdbc-class-work
```



Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)

- `select * from xxx` – антипаттерн
- НЕ рекомендуется к использованию в реальных приложениях
- А значит и в ДЗ
- Представим что есть таблица книг, откуда мы, с использованием “`select *`” выбираем 100 000 записей, размером 100 байт. Т.е. размер набора равен 10 000 000 байт или 9,5 МБ
- А теперь в книгу добавляется ознакомительный фрагмент в 2000 байт, который в нашем приложении никак не используется. Но размер набора, который мы грузим автоматом увеличивается до $100\,000 * (100 + 2000) = 200,3\text{ МБ}$

05

Unit-тестирование Spring JDBC



- Поднимает часть контекста, ответственную за БД. В т.ч. выполняются `schema.sql` и `data.sql`
- По умолчанию создает вначале теста и откатывает в конце транзакцию
- Нужен `@RunWith(SpringRunner.class)` при использовании JUnit4
- Нужен `@ExtendWith(SpringExtension.class)` при использовании JUnit5
- У последних версий Spring Boot `@ExtendWith(SpringExtension.class)` есть внутри `@JdbcTest`, так что его писать не обязательно

```
@DisplayName("Dao для работы с персонами ")
@ExtendWith(SpringExtension.class)
@JdbcTest
@Import(PersonDaoJdbc.class)
public class PersonDaoJdbcTest {

    @Autowired
    private PersonDaoJdbc dao;

    @DisplayName("возвращать заданного пёрсона по его Id ")
    void shouldReturnExpectedPersonById () {
        // Write test here
    }
}
```

Демо

(Еще раз смотрим тест)



Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)

06

Домашнее задание



- Разработать однопользовательское, консольное приложение позволяющее вести каталог книг в библиотеке
- Каждая книга имеет автора и как минимум один жанр (в зависимости от выбранной сложности работы)
- Задание выполняется на базе заготовки, содержащей каркас приложения
- Необходимо "дописать" классы и служебные файлы заготовки, ориентируясь на оставленные в ней подсказки так, чтобы выполнялись условия задания
- Также решение должно удовлетворять нижеуказанным требованиям

- То, что уже было в заготовке, должно остаться неизменным (количество, имена и текущее содержимое классов и файлов, сигнатуры методов и т.д.)
- Можно добавлять код/текст в существующие методы классов и файлы, писать свои приватные методы, а также создавать новые классы
- Вышеуказанные требования не относятся к комментариям, их после реализации нужно удалить

- Использовать Spring JDBC и реляционную базу H2
- Для параметризованных запросов использовать `NamedParametersJdbcTemplate`
- Каждая сущность должна храниться в своей таблице
- Отношение между книгой и автором - многие-к-одному (у книги один автор, но у автора может быть несколько книг)
- Отношение между книгой и жанром зависит от выбранной сложности работы. Многие-к-одному (у книги один жанр, но у жанра может быть несколько книг) или многие-ко-многим (у книги много жанров, и у жанра может быть несколько книг)
- Интерфейс выполняется на Spring Shell. Для сущности книги, предусмотреть операции создания, обновления, удаления, вывода всех книг, а также вывода книги по id
- Предусмотреть операции вывода полного списка сущностей авторов и жанров

- Создание и инициализация схемы БД должно происходить через `schema.sql` + `data.sql`
- Не делать абстрактных или обобщенных сущностей
- Не делать абстрактных или обобщенных Dao
- Не делать двунаправленных связей (в книге автор, в авторе книги)
- Проблема N+1 должна быть решена
- С помощью `@JdbcTest` сделать интеграционные тесты всех методов дао жанров (со встроенной БД)
- Без фанатизма)
- Опционально*:
 - Создание и инициализация схемы БД сделать через систему миграций (Liquibase/Flyway)
 - Отношение книга-жанр, многие-ко-многим (должен использоваться соответствующий вариант заготовки)

- Заготовка для выполнения работы:
 - Базовый вариант:
<https://github.com/OtusTeam/Spring/tree/master/templates/hw05-jdbc-simple>
 - Вариант с опциональным усложнением:
<https://github.com/OtusTeam/Spring/tree/master/templates/hw05-jdbc-hard>
- Задание сдаётся в виде ссылки на pull-request в чат с преподавателем в личном кабинете ОТУС, а не в Telegram
- В pull-request должно присутствовать только то что касается текущей работы. Временные файлы и файлы IDE не должны попадать в PR.
- Инструкция по работе с pull-request-ами находится в документе "Настройка окружения.pdf" в материалах занятия
- Код, написанный в данном ДЗ будет использоваться в дальнейших домашних заданиях всех оставшихся блоков курса

Вопросы?

(ставим “+” если вопросы есть и “-”
если вопросов нет)

**Пожалуйста, пройдите
опрос**

Спасибо за внимание!

Спрингового JDBC!

